Heart Disease Detection – MLOps Assignment

Student Details:

Name: Lokesh Kumar Pandey

Student Id: 2025AE05342

Repository URL: https://github.com/pandeylokesh124/mlops-assignment/tree/main

Video URL : https://drive.google.com/file/d/1qa2KR6goiNEVxu8tykBkHJECo9fw6HLJ/view?usp=drive_link

1. Setup & Installation Instructions

This section outlines the step-by-step setup required to run the data pipeline, train models with MLflow tracking, and deploy the prediction service to a local Kubernetes cluster.

Prerequisites

- **Python:** 3.10+
- **Containerization:** Docker Desktop (with Kubernetes enabled using kubeadm provisioner)
- **Version Control & Shell:** Git, PowerShell 7+ (Windows)
- **MLOps Tools:** MLflow, FastAPI, Kubernetes (kubectl)

Step 1: Clone Repository & Setup Environment

```
# Clone the project repository
git clone https://github.com/pandeylokesh124/mlops-assignment.git
cd mlops-assignment

# Create and activate virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1

# Install required dependencies
pip install -r requirements.txt
```

Step 2: Start Host-Level MLflow Tracking Server

Start the local MLflow server listening on 0.0.0.0 with artifact serving enabled so Kubernetes pods can download models dynamically:

```
$env:MLFLOW_ALLOWED_HOSTS="*"
```

```
mlflow server `
 --backend-store-uri sqlite:///$(pwd)/experiments/mlflow.db `
 --artifacts-destination file:///$(pwd)/experiments/mlruns `
 --host 0.0.0.0 `
 --port 5000 `
 --serve-artifacts `
 --artifacts-host host.docker.internal `
 --workers 1
```

Verification: Access http://localhost:5000 in your web browser to view the MLflow UI.

Step 3: Run Model Training & Registry Pipeline

Execute the training script to train models across Stratified K-Fold CV splits and register the top candidate:

```
$env:MLFLOW_TRACKING_URI="http://localhost:5000"
python experiments/mlflow_runner.py
```

Note: Ensure the top-performing run is registered under the model name Random_Forest_model (Version 1) in the MLflow Model Registry UI.

Step 4: One-Click Automated Deployment to Kubernetes

Run the PowerShell orchestration script to build the Docker image, deploy Kubernetes manifests, and perform health checks:

```
# Set script execution policy for active session
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

# Execute pipeline automation
.\orchestrate.ps1
```

2. Exploratory Data Analysis (EDA) & Modelling Choices

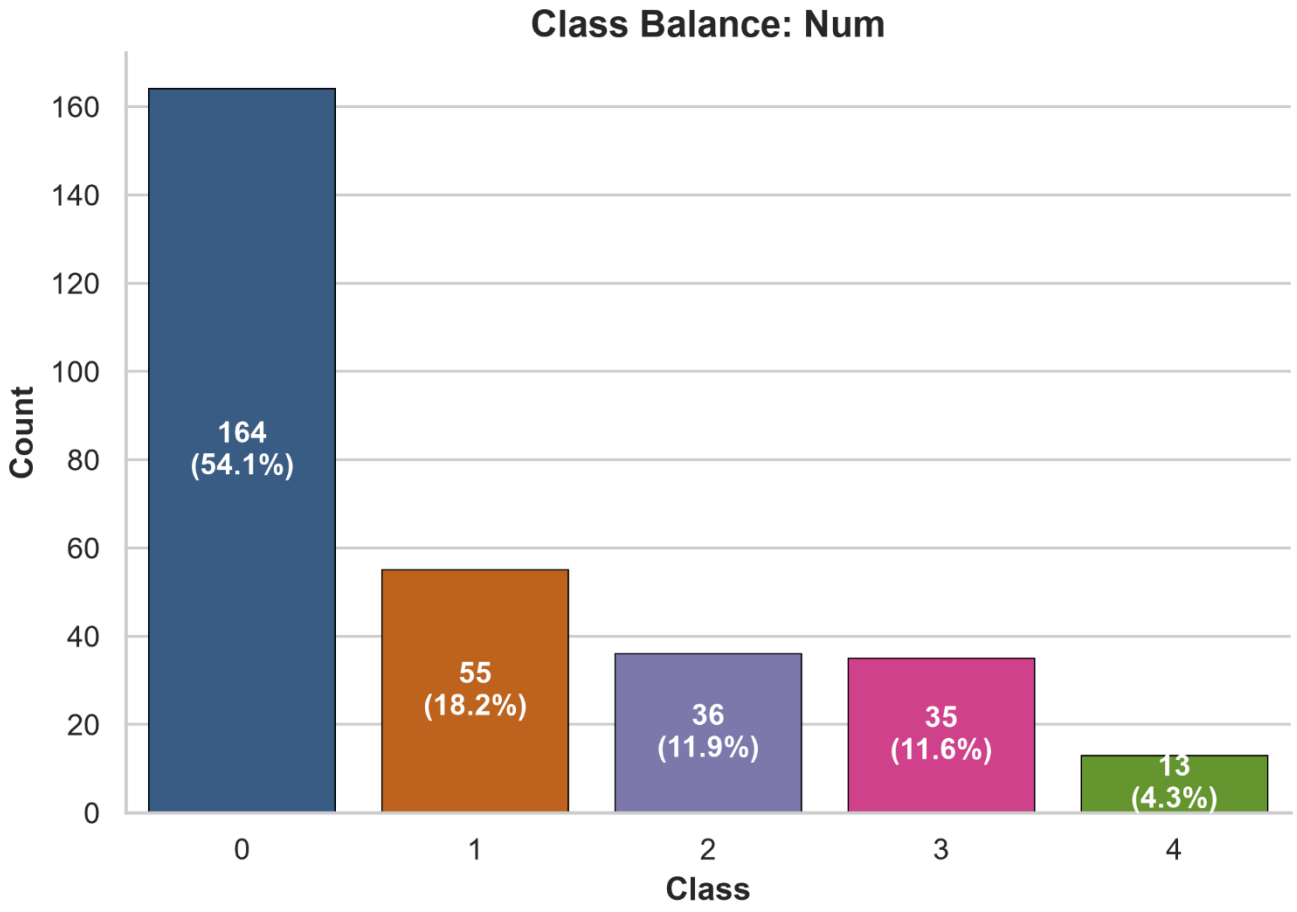
Exploratory Data Analysis (EDA) Summary

The goal of the EDA phase was to understand feature distributions, identify missing data, and determine appropriate preprocessing transformers for patient clinical records.

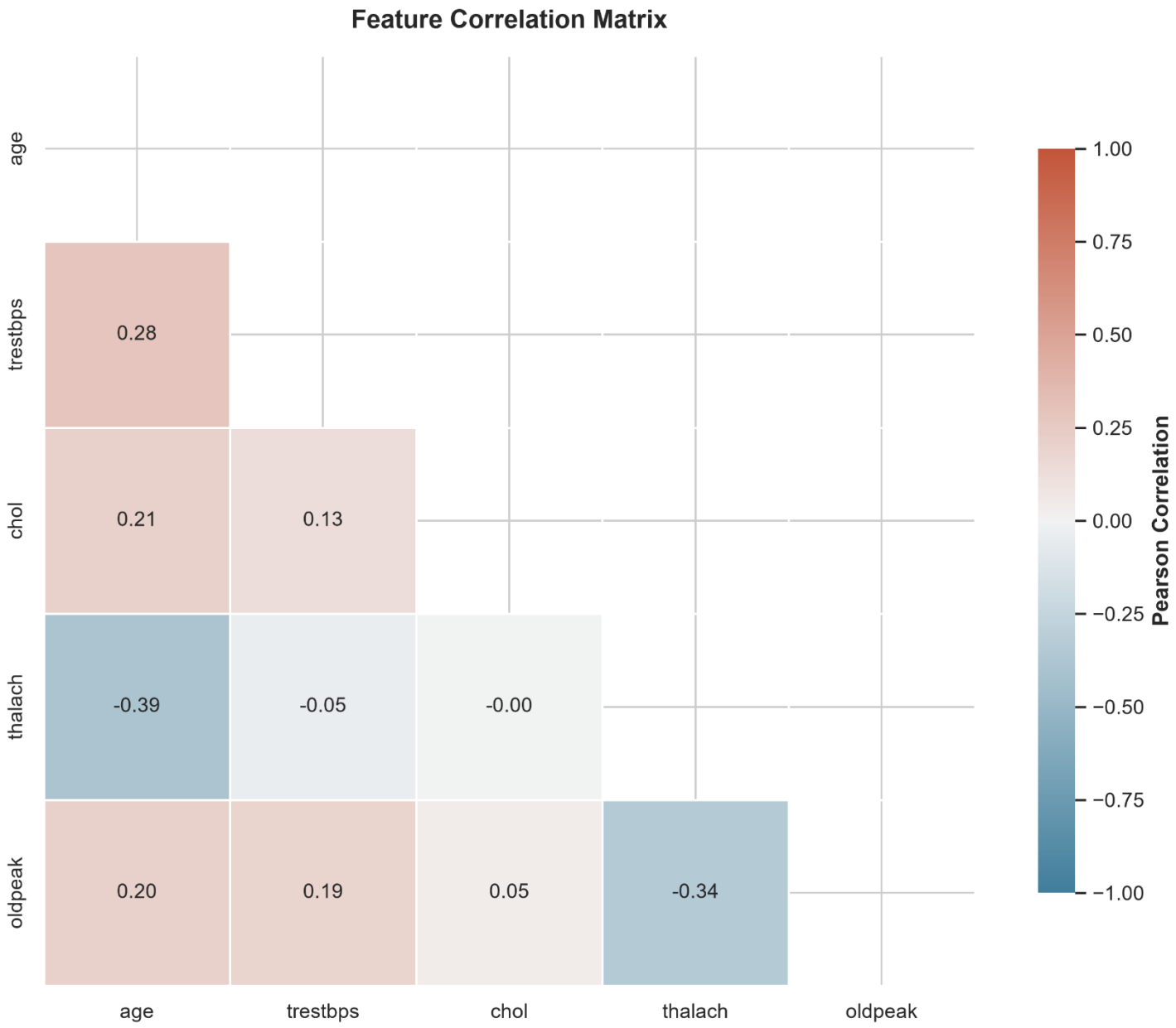
1. **Target Distribution:** Evaluated target variable (target: 0 = No Heart Disease, 1 = Heart Disease) to ensure balanced class distributions and prevent bias during model evaluation.
2. **Feature Associations:**
    - Positive predictors: cp (Chest Pain Type), thalach (Maximum Heart Rate Achieved), and slope.
    - Negative predictors: oldpeak (ST Depression), ca (Number of Major Vessels), and exang (Exercise Induced Angina).

3. **Data Preprocessing & Leakage Prevention:**
    - Bundled all preprocessing into a Scikit-Learn ColumnTransformer.
    - Continuous attributes (age, trestbps, chol, thalach, oldpeak) scaled via StandardScaler.
    - Categorical attributes (cp, restecg, slope, thal) encoded via OneHotEncoder.
    - Preprocessing and classifier steps were wrapped inside a unified Pipeline object to prevent data leakage during Cross-Validation and inference.

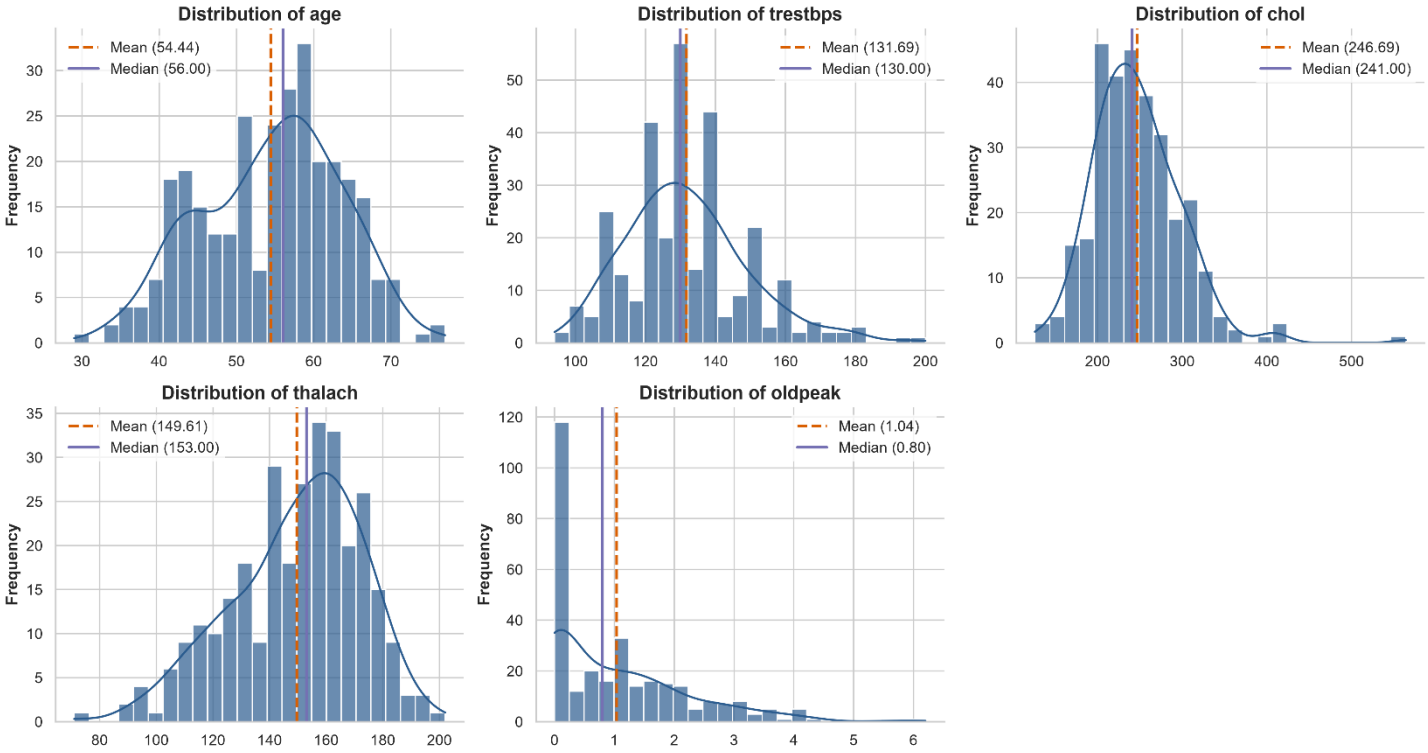
Class balance graph:



Correlation heat map:



Numerical distribution:



Modelling Choices

Multiple object-oriented classifier implementations were trained and compared:

- **Random Forest Classifier:** Chosen as the primary deployment model due to its high accuracy, resilience to tabular noise, and fast CPU inference latency.
- **Support Vector Classifier (SVC):** Trained with Probability Calibration (CalibratedClassifierCV) to output reliable prediction probabilities.
- **XGBoost Classifier:** Used as a gradient-boosted benchmark for cross-validation comparison.

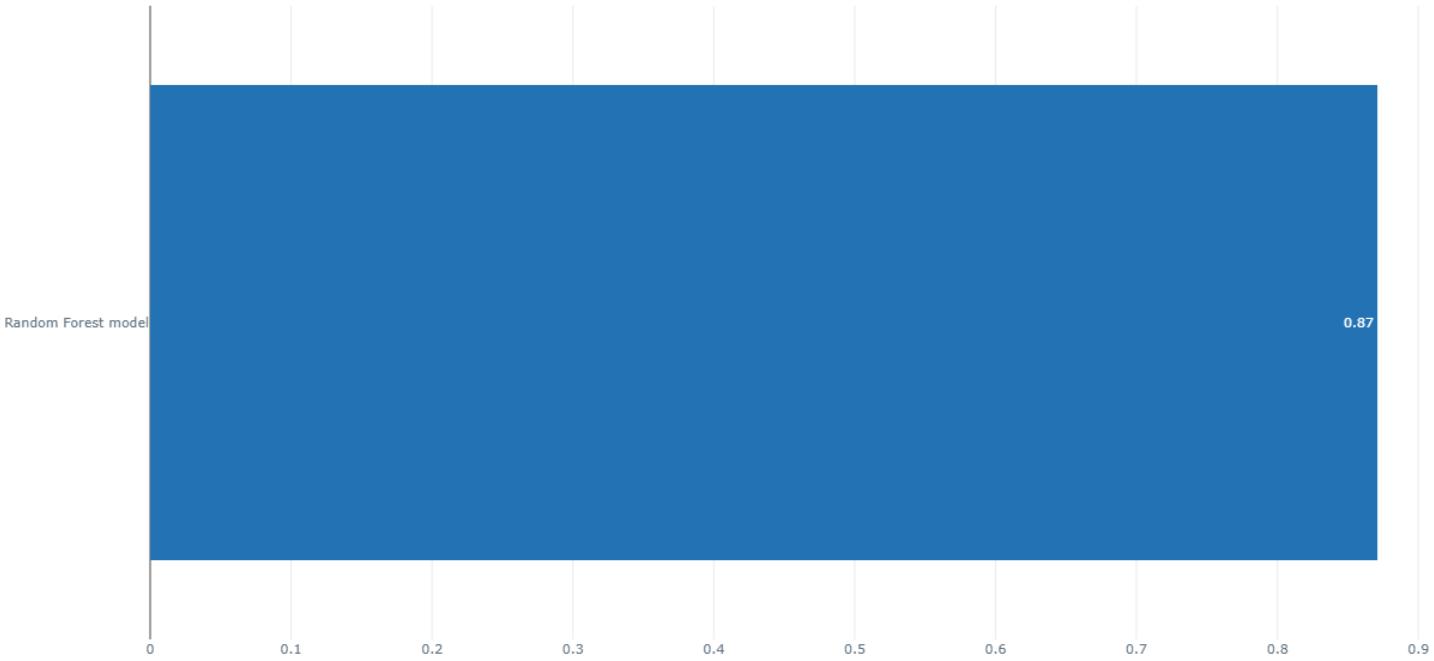
3. Experiment Tracking Summary

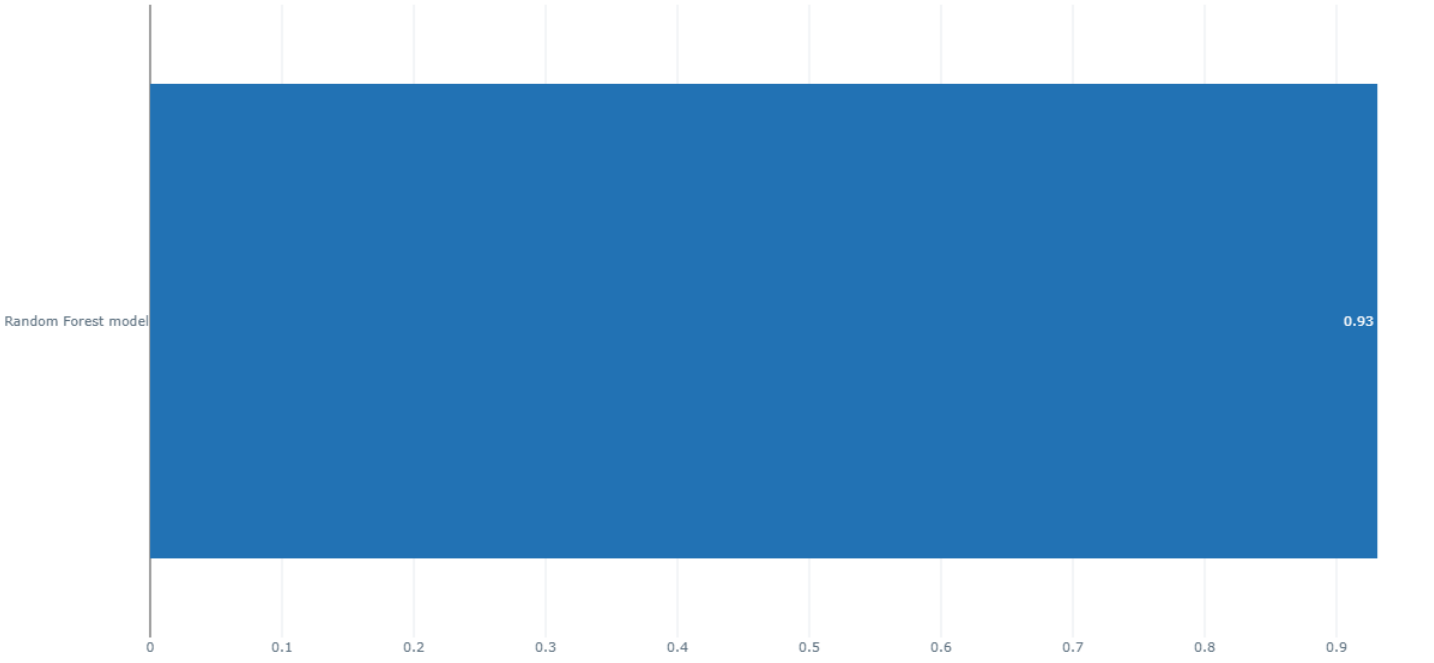
MLflow was leveraged for experiment tracking, artifact logging, and model lifecycle management.

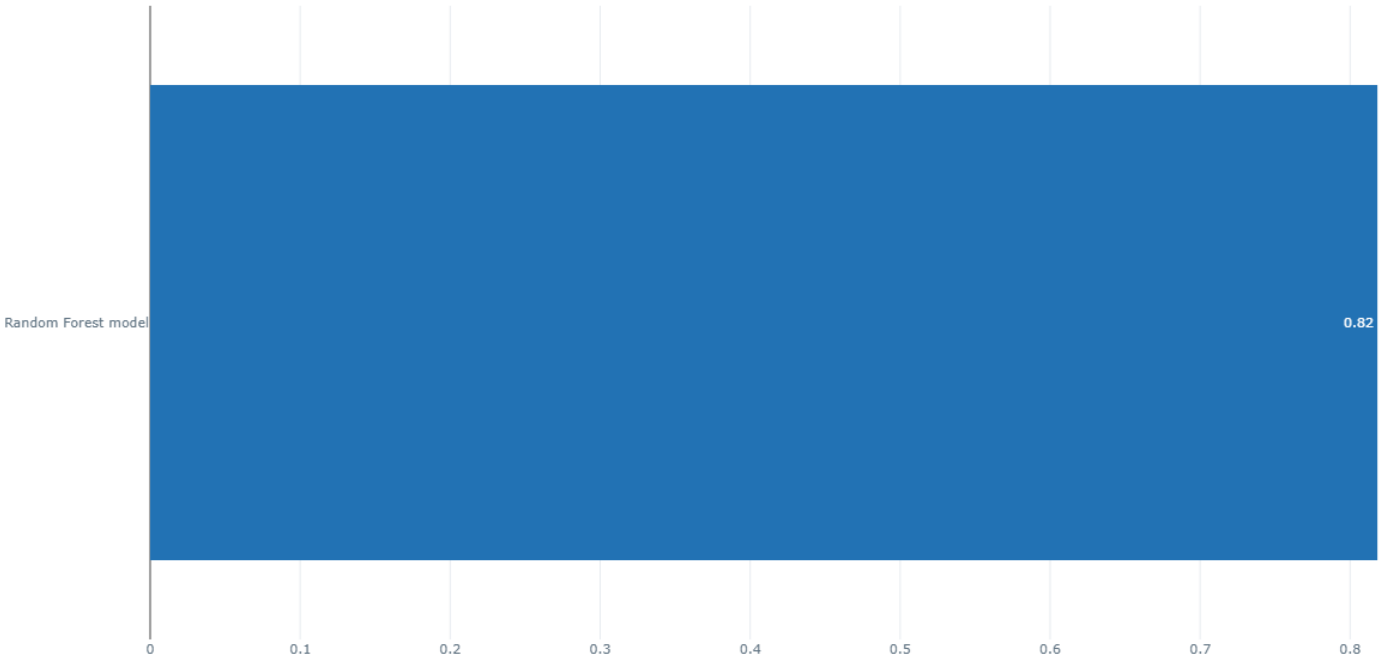
- **Backend Store:** SQLite database (experiments/mlflow.db).
- **Artifact Storage:** Local disk directory (experiments/mlruns) served via HTTP.
- **Tracked Parameters:** n_estimators, max_depth, min_samples_split, learning_rate, C, kernel.
- **Tracked Metrics:** Accuracy, Precision, Recall, F1-Score, and ROC-AUC across Stratified 5-Fold Cross-Validation.
- **Log Artifacts:** Model pipelines serialized in .skops / binary format, confusion matrices, and feature importance charts.
- **Registered Model:** Model Name: Random_Forest_model | Stage/Version: 1.

Experiments Result

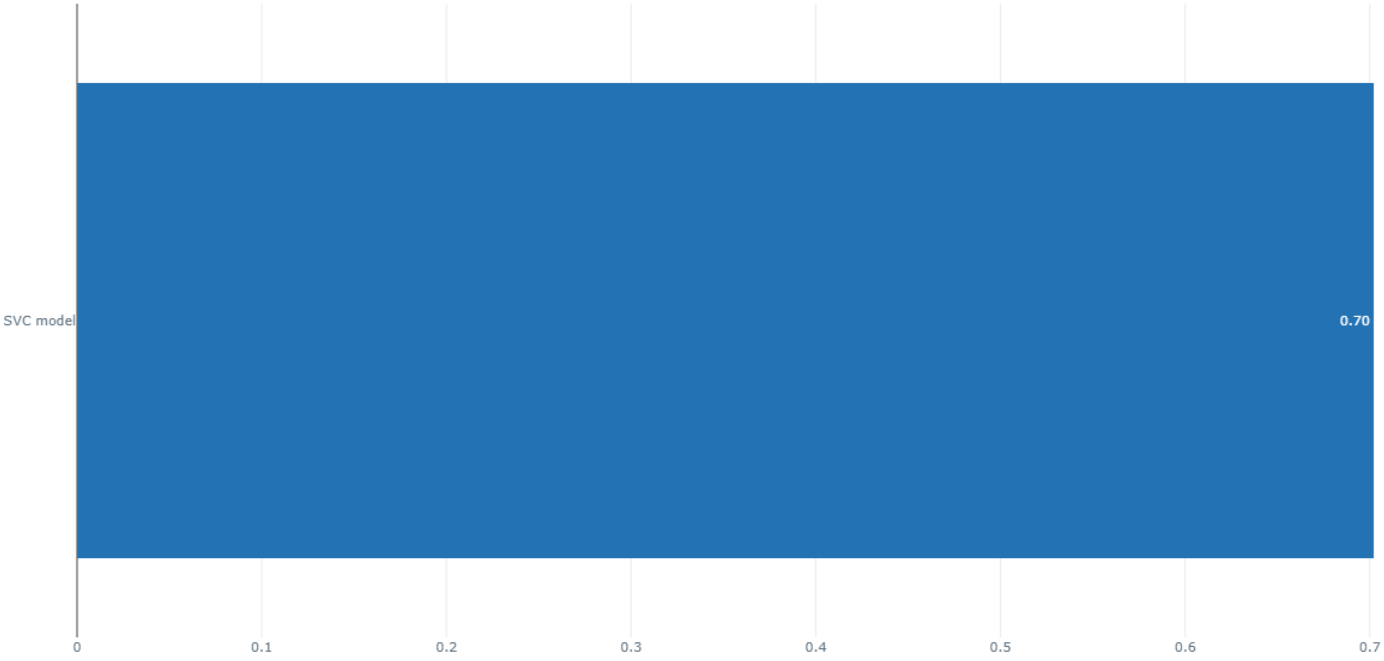
Class 0: Random Forest Model

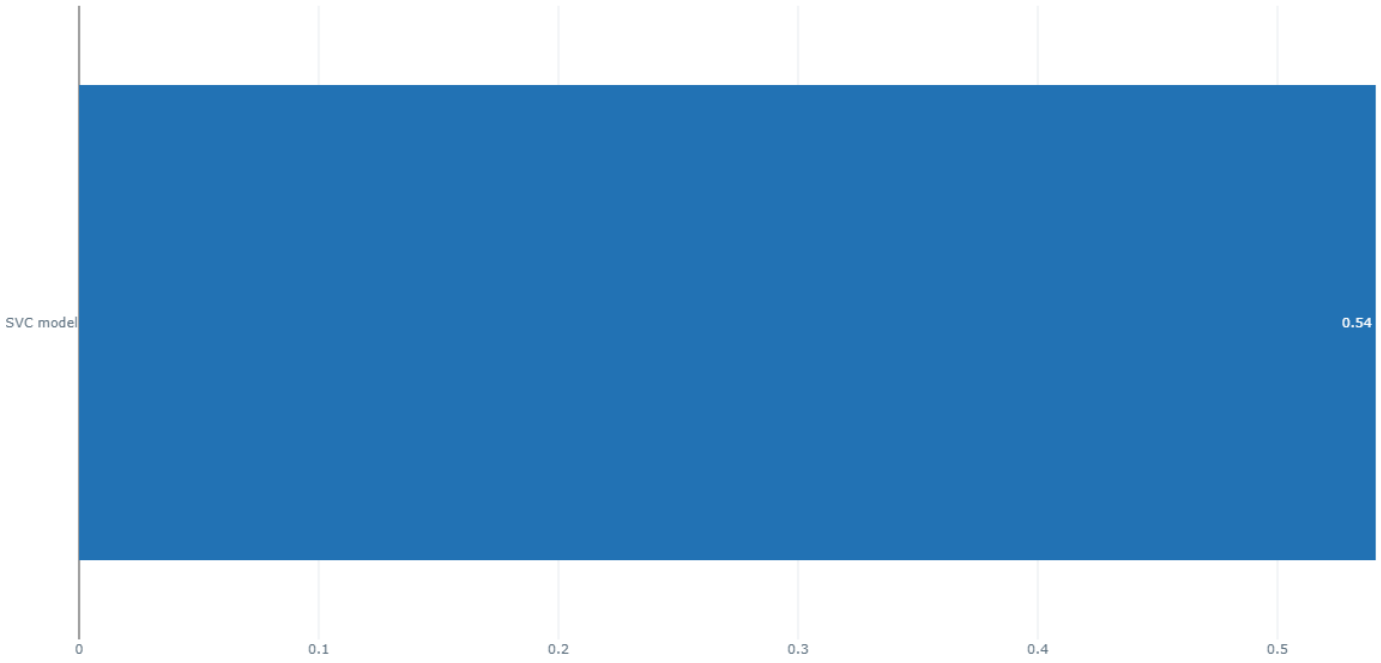

Random Forest model
0.87
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9

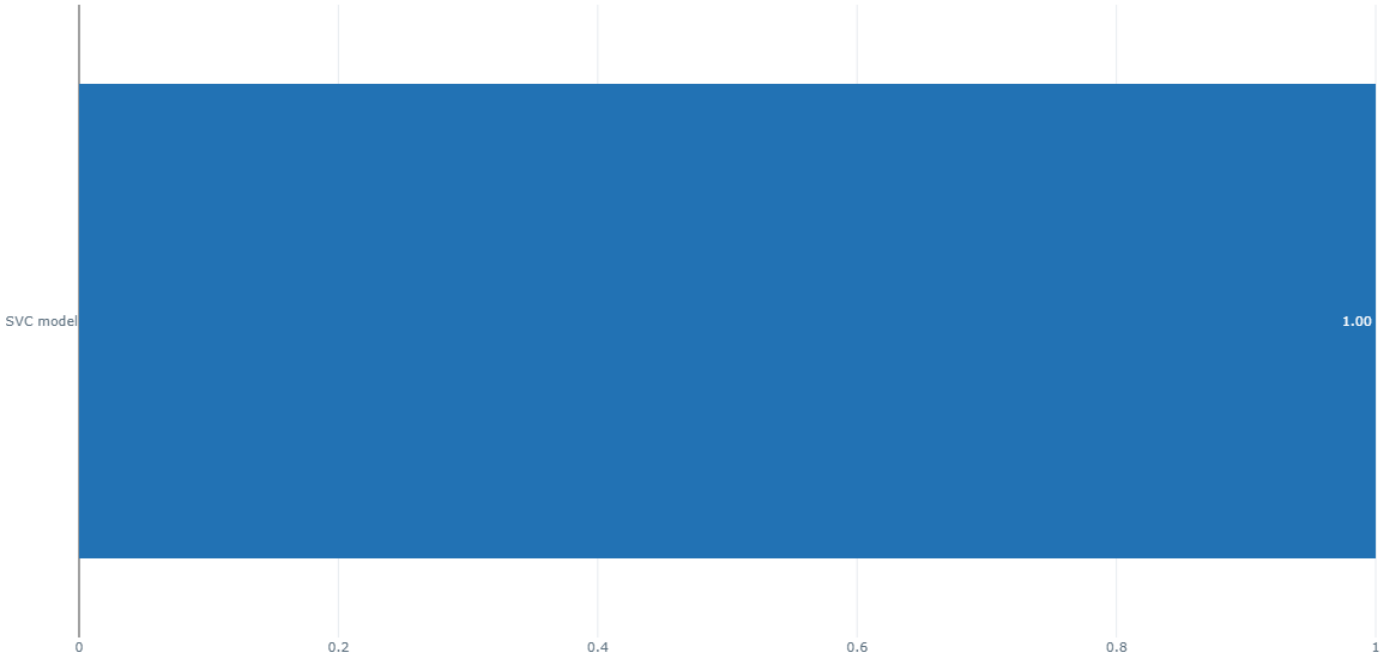

Random Forest model
0.93
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9



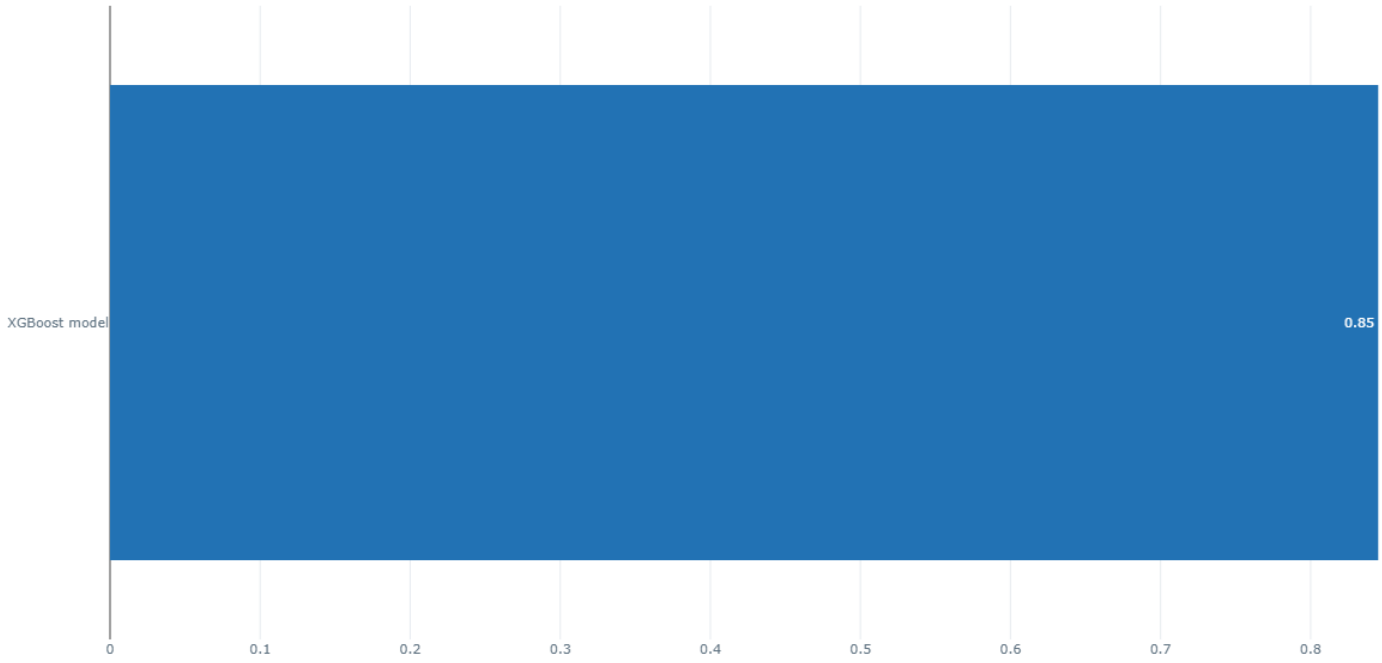
Support Vector Classifier

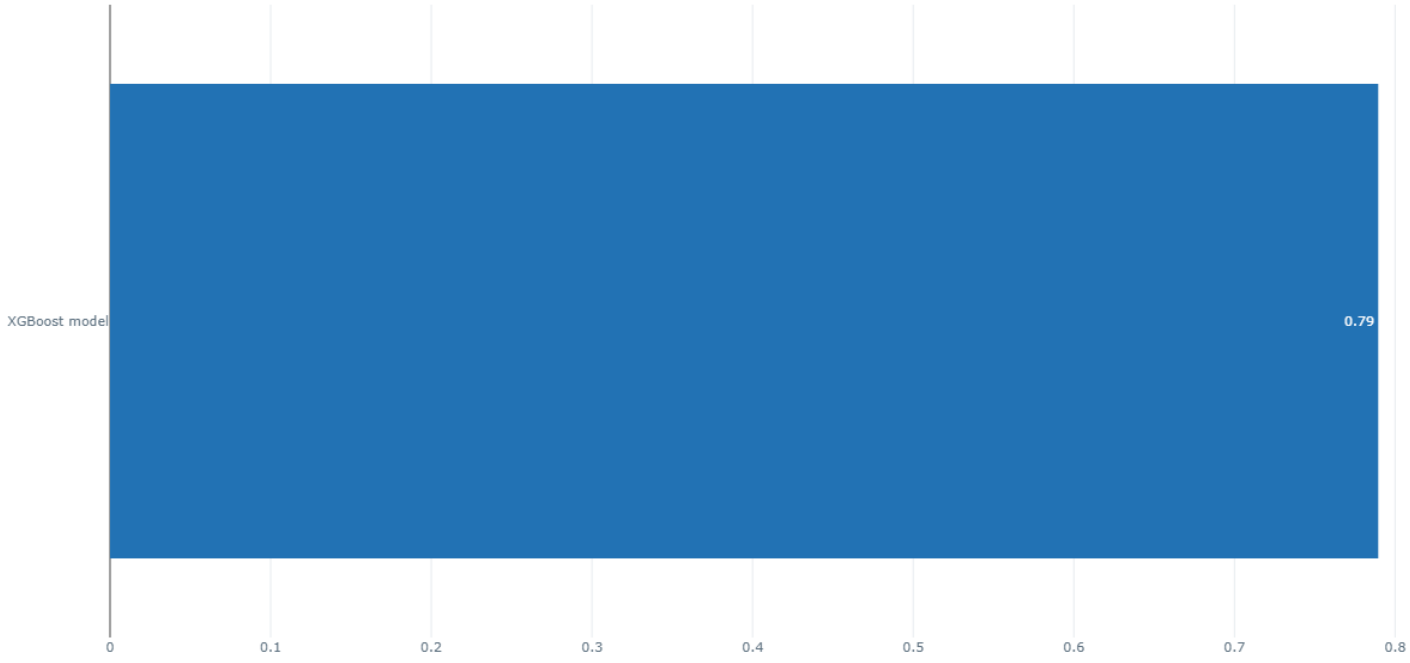


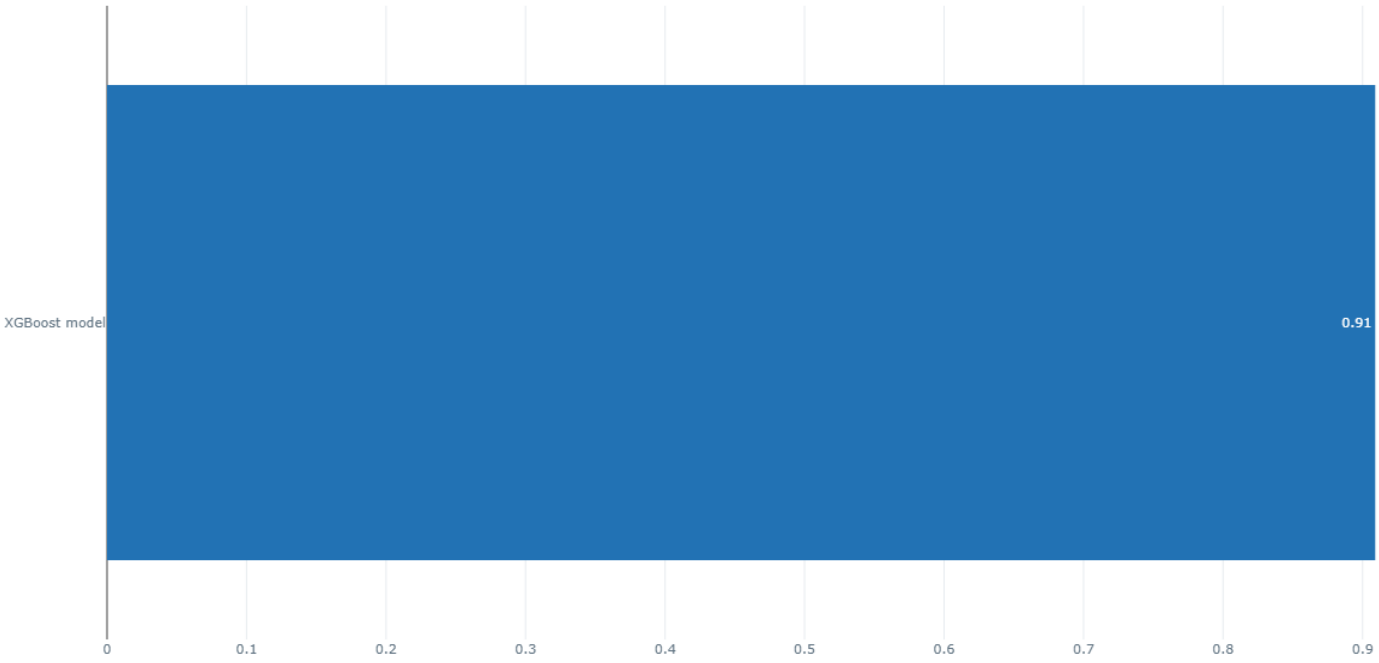




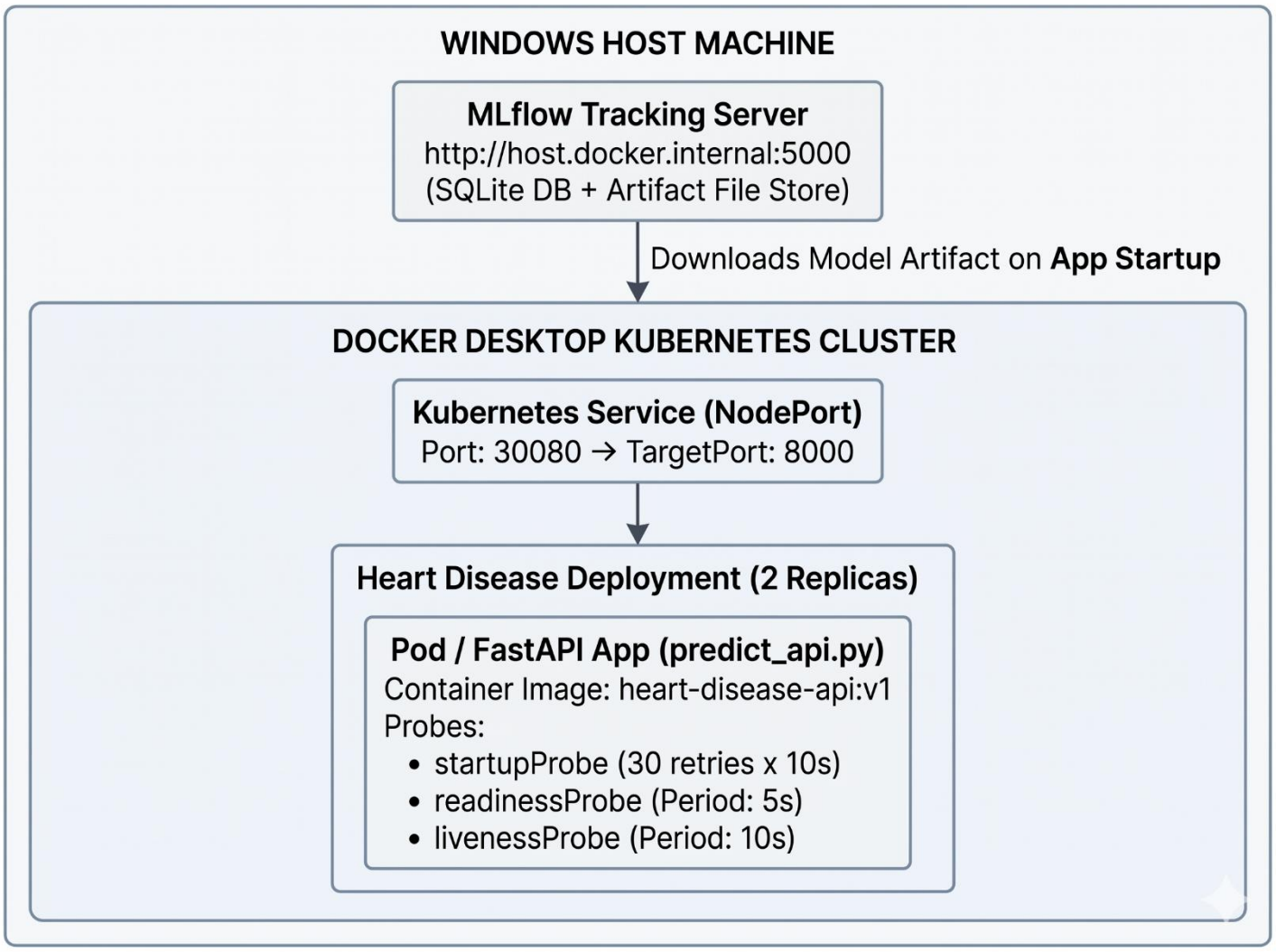
XGBoost:


XGBoost model
0.85
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8


XGBoost model
0.79
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8



4. System Architecture Diagram



5. CI/CD & Deployment Workflow

Containerization & Kubernetes Orchestration

1. **Docker Container:** Built from python:3.10-slim, exposing Uvicorn on port 8000.

2. **Dynamic Model Fetching:** On container startup, FastAPI queries http://host.docker.internal:5000 via MLflow API to download and load Random_Forest_model Version 1 directly into memory.
3. **Resilient Health Probes:**
    - startupProbe: Configured with failureThreshold: 30 and periodSeconds: 10 (giving up to 300 seconds) to allow the container enough time to pull model weights without triggering premature container restarts.
    - readinessProbe & livenessProbe: Actively monitor /health every 5 to 10 seconds once startup completes.

Automated Deployment Verification (orchestrate.ps1)

The automation script performs the following sequence:

1. Validates MLflow server connectivity on port 5000.
2. Builds heart-disease-api:v1 into the local Docker daemon.
3. Applies helm/templates/deployment.yaml and helm/templates/service.yaml.
4. Executes kubectl rollout status to monitor pod readiness.
5. Sends a test health check to http://localhost:30080/health.

API Inference Endpoint Reference (POST /predict)

Request Endpoint: POST http://localhost:30080/predict

Header: Content-Type: application/json

Sample Request Payload:


```
{
  "age": 63.0,
  "sex": 1,
  "cp": 3,
  "trestbps": 145.0,
  "chol": 233.0,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150.0,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
```

}

Sample Response Payload:

{

"predicted_class": 0

}

6. Code Repository Link

The complete codebase, Dockerfiles, Kubernetes manifests, and automation scripts are hosted on GitHub:

- **Repository Link:** https://github.com/pandeylokesh124/mlops-assignment/tree/main

Student Details:

Name: Lokesh Kumar Pandey

Student Id: 2025AE05342

Repository URL: https://github.com/pandeylokesh124/mlops-assignment/tree/main

Video URL : https://drive.google.com/file/d/1qa2KR6goiNEVxu8tykBkHJECo9fw6HLJ/view?usp=drive_link